LUXE & THREAD · DOCUMENTATION

INTERNAL – for developers

Internal 01 · Requirements Review & Gap Fixes

Audience	INTERNAL – developers, tech lead
Purpose	Answers each of the 11 questions in *Project_Fashion_Store.md*: was it implemented, what was fixed, what is still open
Version	2.0 · October 2026

Legend: ✅ done and unit-tested · 🟡 done, needs a run on real infrastructure (database, providers, browser) · ➡️ recommended next step

> **Honest verification note.** Everything in this repository passes the strict TypeScript check, ESLint and **23 automated unit tests** of the business rules (pricing, coupons, sizes, state machine, reports, CSV import, templates, passwords, sessions, signatures). The PostgreSQL SQL, Next.js production build, Razorpay/Shiprocket/e-mail calls and the browser tests could **not** be executed in the authoring environment (no database server, no internet). They are written to the providers' documented behaviour and each has a test/procedure in Internal 11. Run those before go-live.

Summary table

#	Question	Was it there?	Now	Where
1	Header buttons: Home, About, Shop, Women/Men/Kids…	❌ Only Home & Shop	✅ Women · Men · Kids · New Arrivals · Sale · About · Contact + search + bag; mobile menu	`components/Header.tsx`, `/about`, `/contact`
2	Sizes S–XXL + Indian standard	❌ Free-text sizes	✅ Size sets (XS–XXL, kids by age, footwear UK/India, free) + Indian size charts + live stock per size	`lib/sizes.ts`, `/size-guide`
3	Admin: upload stock, change price, discounts, out of stock	❌	✅ Product screen, stock receive/stock-take, CSV refill, festival sale, per-product sale price with dates, coupons, mark out-of-stock, hide/show	`/admin/products`, `/admin/inventory`, `/admin/coupons`
4	Dashboard: sales, history, stock, profit/loss by day/month/year, download	❌	✅ Dashboard + Reports (day/month/year), expenses, CSV + print-to-PDF	`/admin`, `/admin/reports`, `/admin/expenses`

#	Question	Was it there?	Now	Where
5	Notifications: e-mail, SMS, WhatsApp	❌	✅ Outbox + retries; e-mail (SMTP), SMS (HTTP gateway), WhatsApp (Cloud API) – optional channels	`lib/notifications/*`
6	Store customers for marketing; PostgreSQL; product IDs; CSV/image upload	❌	✅ PostgreSQL, customers with **opt-in** consent, campaigns, SKU model, CSV import, photos auto-attached by file name	`db/migrations`, `lib/repos`, `/admin/customers`
7	Open-source only	partly	✅ See open-source table	—
8	All possible testing	unit only	✅ Pyramid: unit, integration, API/contract, e2e, load, security, accessibility	Internal 11
9	Customer cancellation design & flow	❌	✅ Customer cancel / return, auto refund, courier cancel, restock, late-payment refund	Internal 13
10	Day-to-day, interactive, good feel	basic	✅ Search, filters, sort, sale badges, low-stock hints, gallery, live bag prices, coupons, skeleton loaders	storefront pages
11	"Rock solid", market standard	partial	✅ Hardening list below; 🔜 items flagged	Internal 14

1 · Header navigation

Fix: department-based navigation (Women, Men, Kids), New Arrivals, Sale, About, Contact, search box and bag with live count. On phones the links scroll horizontally under the logo. Footer repeats the links and adds the Indian size guide and policy pages. **Why:** shoppers browse a fashion store by *who it is for* first. **Next:** mega-menu per category once the catalogue grows.

2 · Sizes (S, M, L, XL, XXL) and Indian standard

Design. A product has a **size set**: `apparel` (XS S M L XL XXL), `kids` (2-3Y … 12-13Y), `footwear` (UK/India 5–11) or `free`. Each size is a **variant** with its **own stock** (`product_variants`). The product page shows every size, disables sold-out ones, and warns "only 2 left".

Indian standard. `/size-guide` lists body measurements in inches for women (India size = bust 32–42), men (India size = chest 36–46), bottoms (waist), kids (age + height) and footwear (UK/India/EU/US + foot length). Values are typical Indian charts; **have your tailor confirm them against your own garments** (brands differ by ±½ inch).

Size	Women (bust in)	Men (chest in)	Waist bottoms (in)
XS	32	36	–
S	34	38	28–30
M	36	40	32

Size	Women (bust in)	Men (chest in)	Waist bottoms (in)
L	38	42	34
XL	40	44	36
XXL	42	46	38

Why variants: stock is per size, so the shop can never sell the last "M" twice (enforced by `CHECK (stock >= 0)` plus a conditional `UPDATE`).

3 · Admin: stock, price, discounts, out of stock

Task	Where	Notes
Add a product	Products → + *Add product*	SKU, prices, sizes with starting stock
Receive new stock	Product → *Stock by size* → **Add**	Recorded as `receive` in `stock_movements`
Stock-take (fix a count)	same → **Set**	Recorded as `adjust` with the difference
Refill many at once	Inventory → CSV `variant_sku, add_qty, note`	Validated first ("Check file only")
Change price / MRP / cost	Product → Details	Old orders keep their frozen price
Discount one product	Product → *Discount price* + dates	Switches on/off automatically
Festival sale on many	Products → *Festival sale* (department/category, %, dates)	Reversible with **Remove sale**
Coupon codes	Coupons	percent/flat, minimum, cap, dates, usage limit
Mark out of stock	Product → **Mark whole product out of stock**	Sets every size to 0 (logged)
Hide a product	Product → Visibility	Keeps history

Every change writes to `audit_log` (who/what/when).

4 · Dashboard & reports

Dashboard: orders to ship, return requests, low-stock/out-of-stock counts, failed messages, today/month/year sales and profit, last-7-days bar chart, best sellers, low-stock list. **Reports (owner):** presets (today … last year) or custom dates, grouped by **day / month / year**, with: gross sales, coupon discounts, refunds, net sales, GST, net sales ex-GST, cost of goods, gross profit, payment fees (estimated %), courier cost, expenses, **net profit or loss**. Downloads: summary CSV, order-wise CSV, stock CSV, orders CSV, customers CSV; **Print / Save as PDF** button. **Profit formula and its limits** are in Admin Dashboard Guide. Cost price per product must be entered for profit to be meaningful.

5 · Notifications

Events that send messages: order confirmed, payment failed *(template ready)*, shipped, delivered, cancelled, refund processed, return updates. Channels: e-mail (always when SMTP is set), SMS (if enabled), WhatsApp (if enabled **and** the customer opted in). Design: Internal 12.

6 · Customers, PostgreSQL, product IDs, CSV

- **PostgreSQL 16** (open source) – schema in `db/migrations/001_init.sql`; table-by-table reference in Internal 03.
- **Customers** are stored at checkout with three separate, **un-ticked-by-default** consent flags (e-mail/SMS/WhatsApp). Marketing goes only to opted-in customers and every message carries an unsubscribe link.
- **Product IDs.** `SKU` = style code (e.g. `LT-KMB-002`) – permanent. Sizes are `SKU-SIZE`. A **new arrival** = new SKU. A **refill** = more stock on the *same* variant (never a new product). Price/discount changes edit the product.
- **Better than one big CSV?** We use **two small CSVs + SKU-named photos**: *Products CSV* (new products / price & detail changes – never overwrites stock), *Stock CSV* (refills, additive), and photos named `LT-KMB-002_1.jpg` that attach automatically. This avoids the classic mistake of a product sheet accidentally resetting stock.

7 · Open source only

Need	Tool	Licence
Framework	Next.js, React	MIT
Language / styling / validation	TypeScript, Tailwind CSS, Zod	Apache-2.0 / MIT
Database	PostgreSQL 16	PostgreSQL Licence
DB driver	node-postgres (`pg`)	MIT
E-mail	Nodemailer + any SMTP (Postfix, Mailu, or a provider's SMTP)	MIT-0
Web server / process	Nginx, PM2, Node.js	BSD / AGPL (PM2 used as tool) / MIT
TLS	Let's Encrypt (certbot)	free CA
Testing	Node test runner, Playwright, k6, OWASP ZAP	MIT / Apache / AGPL / Apache
Docs	Graphviz, Pandoc, Chromium (PDF)	EPL / GPL / BSD
Backups	`pg_dump`, cron, rclone/aws-cli	PostgreSQL / Apache

Not open source but unavoidable (services, not software you install): Razorpay (payments), Shiprocket (couriers), an SMS gateway (TRAI/DLT rules), WhatsApp Business Cloud API (Meta), domain registrar, cloud host. Alternatives are listed in Internal 12.

8 · Testing

See Internal 11: 23 unit tests pass; 5 database integration tests; 7 Playwright scenarios (desktop + mobile); a k6 load script; security, accessibility and performance procedures; plus a manual provider test checklist with expected results.

9 · Cancellation

Customer self-service on the order page: instant cancel (unpaid / COD not shipped), cancel + automatic refund (paid, not shipped), courier cancel then refund (booked, not picked up), return request after delivery (7 days). Edge cases: payment arriving **after** cancellation is refunded automatically; refund failures are recorded and retryable by the owner; Shiprocket failure never leaves the order half-cancelled. See Internal 13.

10 · Experience

Search, department/category/size/price filters, sort, "in stock only", sale badges with % off and MRP strike-through, "only N left", multi-photo gallery, related products, live bag with server prices and coupon, clear stock problems in the bag, order tracker, skeleton loaders, mobile-first layout, size guide, PIN-code delivery check. ➡️ Wishlist, recently viewed, product reviews, "notify me when back in stock".

11 · "Rock solid" – what was tightened

Area	Done
Money	Server-side pricing; stock reserved inside the order transaction; DB refuses negative stock; coupon usage counted atomically
Payments	Signature verify + webhook; idempotent; late-payment auto-refund; refund API; unpaid orders expire in 30 min
Data	PostgreSQL constraints (CHECK/UNIQUE/FK), audit log, stock diary, order event log
Auth	Admin users table, scrypt password hashes, roles (owner/staff), signed HttpOnly cookie, login rate limit, re-check on every action
Input	Zod everywhere, magic-byte image checks, CSV formula-injection guard, parameterised SQL only
Privacy	Opt-in consent, unsubscribe, order links need a secret token, redacted logs
Operations	Health endpoint with DB check, background jobs, backups with restore drill, runbooks
Headers	Security headers + CSP in report-only mode

➡️ **Still recommended:** shared rate-limit store (Redis), WAF/CDN (Cloudflare), error monitoring (Sentry), GST invoice PDFs, 2FA for admin, automatic abandoned-cart e-mails, inventory reservation timeout for slow UPI, CI pipeline (GitHub Actions) running `typecheck`, `lint`, `test`, `build`.

Suggested improvement backlog (priority order)

1. Run the full test plan on a staging server; fix findings. 2. GST invoice PDF + numbering. 3. Admin 2-factor login. 4. Wishlist & back-in-stock alerts. 5. Reviews & ratings. 6. Redis rate limiting + job queue. 7. Search upgrade (PostgreSQL full-text → Meilisearch). 8. Multi-warehouse and barcode scanning for stock.